

DRAINSinkhorn: Safe Elimination for Batched Entropic Optimal Transport*

Xinyang Wen

Abstract

Fast OT backends reduce the cost of one Sinkhorn update, but fixed-width batched execution still runs at full width until its slowest problem finishes. We present DRAINSinkhorn, an execution layer for batched entropic optimal transport that dynamically removes completed problems and narrows subsequent updates. It starts from standard batched execution of independent Sinkhorn problems. After an alternating scaling cycle, one marginal is current by construction; a batched screening of the other marginal filters the problems that require the two-marginal verifier. Problems that pass the stopping rule are removed from every per-problem tensor, while unfinished problems continue.

We develop a hardware-aware performance model. For first-passing iteration numbers $n_1, \dots, n_W$ and active batch width $A_k = |\{t : n_t \geq k\}|$, static execution performs $Wn_{\max}$ problem updates, whereas dynamic compaction performs exactly $\sum_k A_k = \sum_t n_t$. If $c_k(a, \xi_k)$ is the measured update cost at width a , the exact matched-path runtime difference is the update-time reduction $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ minus incremental control overhead. Different completion depths create removable work, while measured kernel time at each active width determines the runtime benefit. On the measured A100/Triton configuration, wave quantization makes update time step-wise: for $n = 1024$, widths one through six have similar cost, whereas larger problems respond to width reduction earlier.

Matched fixed-width/dynamic-compaction comparisons give $1.746\times$ on the four-worker MetroPT-3 endpoint, $1.054\times$ for ImageNet-32 Sinkhorn time, and up to $3.110\times$ in the MetroPT-3 scale-out study. An OTT-JAX realization gives $1.366\text{--}1.581\times$ across three tolerances. When one initialization is used for multiple problems, matched packed-Triton dynamic compaction gives $1.421\times$ and removes 30.62% of problem updates. In Packer19, a matched logical-masking control isolates compaction, and the gain vanishes when all problems finish at the same verification round. All reported consumer and training-quality checks pass.

*Code: github.com/cis-acorniticacid/DrainSinkhorn

1 Introduction

GPU implementations of entropic optimal transport (EOT) have become markedly better at executing a single Sinkhorn update. Matrix-free operators, tiled reductions, and fused streaming kernels reduce memory traffic and avoid materializing the transport matrix [4, 5]. Many workloads, however, prepare a batch of couplings for predictive maintenance, scientific state transitions, or mini-batch training. Once the inner update is fast, total cost also depends on how long each problem remains in that batch.

Different problems usually reach the stopping tolerance at different iterations. A static batch runs until the problem requiring the most iterations finishes: completed problems continue to occupy state, verification bandwidth, and kernel lanes. A logical mask freezes their state but does not shrink the tensor seen by a fused kernel. The updates between each problem’s completion and the end of the batch are wasted OT work.

We introduce DRAINSinkhorn, an execution layer that removes this padding. It starts from standard batched execution of independent Sinkhorn problems and removes a problem after it passes verification used by the selected solver. A full alternating cycle makes the newly updated marginal current in exact arithmetic, so the other marginal provides a cheap batched screen. Lanes that pass the subsequent two-marginal verifier are removed from potentials, marginals, support descriptors, and scratch buffers. The next backend update runs at the width of the unfinished set.

This design separates kernel optimization from batch execution. The inner backend determines the cost of one stabilized update; DRAINSinkhorn changes the number and width of updates executed across a batch. We implement the execution layer with a packed Triton kernel, OTT-JAX, and PyKeOps. The packed Triton path additionally uses the current marginal in a preliminary screen; the OTT-JAX and PyKeOps paths remove completed problems with independent backend mechanisms.

For problem t , let n_t be the batched-iteration number of the first configured verification round at which it passes the two-marginal verifier. Fixed-width execution performs $Wn_{\max}$ problem updates, whereas dynamic compaction performs exactly $\sum_t n_t$. This identity depends only on the observed iteration numbers. Removable work exists whenever a batch contains both completed and unfinished

problems before an update. If $c_k(a, \xi_k)$ denotes the measured cost of update k at width a , the wall-clock saving is $\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)]$ minus incremental control overhead. The completion depths and width-dependent update times therefore determine the runtime benefit.

GPU update time can change at scheduling-wave boundaries rather than in proportion to the number of problems. The packed Triton update launches a width-dependent number of thread blocks; reducing A_k saves time when the grid requires fewer waves. We measure this relation with a width sweep.

Our contributions are:

1. **Dynamic compaction during batched Sinkhorn.** Starting from standard fixed-width batching and the selected solver’s stopping rule, we dynamically remove completed problems from all per-problem state. Each subsequent kernel executes at a width of the unfinished set.
2. **Hardware-aware performance model.** An update-count identity measures removed work. A matched runtime decomposition uses measured update time at each active width and includes control overhead.
3. **Component analysis.** Matched controls separate batched verification, Sinkhorn-specific screening, logical masking, and dynamic compaction. Grouping experiments that hold total iteration counts fixed and width sweeps identify when dynamic compaction reduces runtime.
4. **Evaluation across backends and tasks.** MetroPT-3, Packer19, and ImageNet-32 evaluate the packed Triton path; OTT-JAX and PyKeOps test transfer on ImageNet-32 and A2D2. The evaluation reports runtime, residuals, finite-precision decisions, consumer outputs, and training quality.

2 Setting and Prior Systems

Entropic optimal transport. For positive unit-mass marginals $a \in \mathbb{R}_+^n$ and $b \in \mathbb{R}_+^m$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij} (\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \quad (1)$$

With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \text{diag}(u)K \text{diag}(v)$ [3]. Coordinate, relaxed, low-rank, and Newton methods reduce arithmetic or iteration count inside one solve [1, 8, 14, 15]. Fused streaming implementations rewrite stabilized updates as LogSumExp reductions and reduce memory traffic [18]. DRAIN SINKHORN keeps the chosen inner update and changes the set of problems that receive the next one.

Initialization and repeated couplings. Carry, analytic extensions, and learned dual proposals can reduce a new problem’s starting defect [2, 16]. Large-coupling

Flow Matching also uses soft c -transforms when supports change [21]. These mechanisms change the numbers of iterations required by the problems. DrainSinkhorn accommodates both regimes: the formal Packer19 attribution uses the same fixed proposal in every arm, and the ImageNet-32 training experiment uses independent cold-start couplings.

Dynamic batching and batched solvers. Automatic batching tracks the program point of each member in a data-dependent program [13]. Orca schedules autoregressive inference one iteration at a time [20]. Outside OT, Ginkgo’s batched CG provides a related execution precedent: each uniform independent linear system exits once its residual reaches the stopping tolerance, and later iterations skip converged systems inside the fused solver [7]. jaxipm redesigns IPOPT around heterogeneous iteration fusion and replacement of completed problems [17]. Within OT, POT’s `solve_batch` parallelizes independent OT problems represented by batched cost matrices, and OTT-JAX uses JAX transformations to vectorize multiple Sinkhorn solves [4, 6, 12]. Fixed-width multi-problem Sinkhorn execution is the starting point for our work. We target the subsequent removal of completed EOT problems while retaining online pairwise interactions instead of materializing one cost tensor per problem.

DrainSinkhorn addresses the numerical structure of EOT batches. Alternating scaling permits a preliminary screen using the current marginals; the two-marginal verifier is evaluated on problem outputs; dynamic compaction must preserve the batch-element correspondence of potentials, marginals, supports, and scratch state. Generic masking leaves those EOT-specific state transformations and the width unchanged. Table 1 compares the batched object, completion signal, and execution action of prior systems. Our OTT-JAX and PyKeOps variants remove completed problems, whereas the packed Triton realization also performs a preliminary screen using the current marginals.

OT consumers. OTT provides a broad transport API [4]. Multisample Flow Matching consumes an OT coupling to select training pairs [11]; these couplings can be prepared independently of network optimization [21]. We evaluate the complete path from the OT coupling to the downstream consumer for industrial endpoints and carry the same execution path through a fixed-update feature-space OT-FM training pipeline.

3 DrainSinkhorn

3.1 Problem: wasted lanes in a fixed-width batch

Consider a window of W independent EOT problems of the form in equation (1). A batched backend applies one Sinkhorn cycle to every lane in parallel. The problems

Table 1: Fixed-width multi-problem execution is established in OT software. DrainSinkhorn’s comparison point is the action taken after a problem satisfies its completion rule.

System	Batched numerical object	Completion signal	Execution action	Output check
POT <code>solve_batch</code> [6, 12]	Independent OT problems with batched cost matrices	Solver stopping criterion	Fixed-width parallel solve	Solver residual criterion
OTT-JAX <code>vmap</code> [4]	Vectorized independent Sinkhorn problems	Per-solve stopping or a shared iteration budget	Fixed-width vectorized execution	Solver error function
DRAIN-SINKHORN	Positive EOT problems with compatible batched state	A preliminary screen using the current marginals, followed by the two-marginal verifier	Removes passing lanes from potentials, supports, marginals, and scratch buffers	Current row and column residual tolerance

need not pass the stopping rule at the same iteration, yet conventional fixed-width execution retains all W lanes until the slowest problem finishes. Earlier completions therefore continue to occupy positions in later kernels.

DrainSinkhorn stops sending completed problems to later kernels. It retains each problem’s EOT objective, Sinkhorn update, stopping rule, backend, and downstream consumer; its execution layer changes which problems receive the next update.

For problem t , let n_t be the batched iteration of the first configured verification at which the problem passes. Before update k , the required active batch width is

$$A_k = |\{t : n_t \geq k\}|.$$

Fixed-width execution always presents width W to the backend. DrainSinkhorn instead presents width A_k , physically removing each problem after its verified completion.

3.2 Method: screen, verify, and compact

DrainSinkhorn repeats four operations. First, it advances every active problem by one batched Sinkhorn cycle. Second, at the configured verification interval, it computes a cheap one-marginal screen for all active lanes. Third, it applies the backend’s full two-marginal verifier only to screen-positive lanes. Fourth, it emits every verified output and compacts all corresponding per-problem state. The next update then runs on the smaller active batch. The verification interval is the number of batched iterations between consecutive screen-and-verify rounds.

3.3 Why the screen preserves the release rule

Let F_t denote one complete Sinkhorn cycle for problem t , including its marginals, kernel operator, and numerical representation. Standard batching adds a leading problem dimension but no cross-problem reduction. In ideal arithmetic,

$$\tilde{F}(\text{pack}(z_1, \dots, z_W)) = \text{pack}(F_1(z_1), \dots, F_W(z_W)). \quad (2)$$

Filtering the problem dimension therefore preserves the update applied to each remaining problem.

For one problem, a Sinkhorn cycle is

$$u^{k+1} = a \oslash (Kv^k), \quad v^{k+1} = b \oslash (K^\top u^{k+1}), \quad (3)$$

where $\oslash$ denotes elementwise division. Immediately after the second leg, exact arithmetic gives

$$v^{k+1} \oslash (K^\top u^{k+1}) = b. \quad (4)$$

The column marginal is current by construction, whereas the row marginal $u^{k+1} \oslash (Kv^{k+1})$ can remain outside tolerance. DrainSinkhorn batches this row-residual calculation across the active lanes. A problem is screen-positive when its value $\hat{r}_{t,k}^{\text{row}} \leq \tau_{\text{screen}}$.

Screen-positive status triggers the full verifier. The implementation recomputes both marginals and releases a problem when

$$r_{t,k} = \max\{\|\pi_{t,k} \mathbf{1} - a_t\|_1, \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1\} \leq \tau_{\text{stop}}. \quad (5)$$

A rejected problem remains active. A false-positive screen adds an unnecessary verification, while a false-negative screen delays compaction; neither screen outcome can release a problem by itself. The component tests in section 5 replay both cases and evaluate the candidates with the same two-marginal verifier.

3.4 Physical compaction and matched controls

After a problem passes, DrainSinkhorn writes its output and compacts every per-problem object: source and target descriptors, marginals, dual potentials, centered shifts, log weights, and scratch buffers. Compaction must preserve the alignment of these objects so that Equation (2) continues to apply to the remaining problems.

We separate three execution modes. *Fixed-width* execution updates all W lanes through the common final boundary. *Logical masking* records per-problem completion and freezes completed states but keeps width W . *Dynamic compaction* removes verified completions and runs subsequent kernels at width A_k . Fixed-width versus dynamic compaction measures the combined effect of release and width reduction. Logical masking versus dynamic compaction (LM/DC) holds the observed release decisions fixed and isolates the effect of physically narrowing the batch.

Equations (2) and (4) state the ideal-arithmetic invariants. The experiments separately test fixed-work parity, achieved residuals, near-threshold decisions, application outputs, and the timing effects of screening, verification, and compaction.

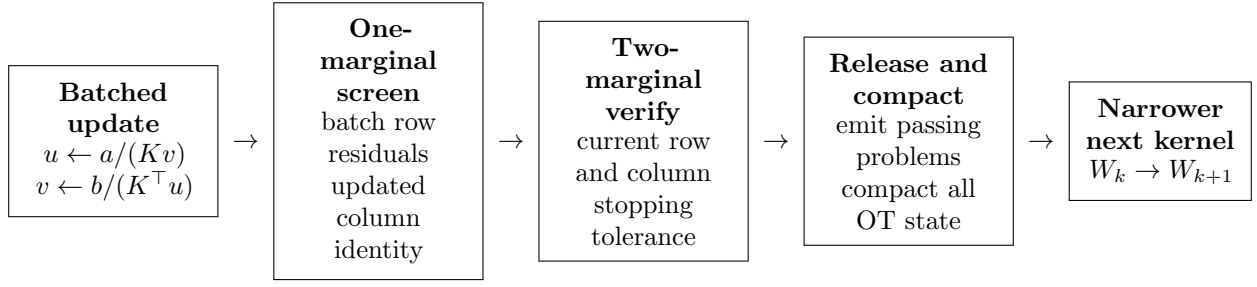


Figure 1: DrainSinkhorn’s execution loop. The one-marginal screen selects candidates for full verification. A problem leaves the batch after passing the two-marginal stopping rule, after which every associated tensor is compacted before the next update.

3.5 Execution boundary and fallback

The comparisons fix the execution mode, initial width, verification interval, proposal, tolerance, and fallback policy before timing. Iteration limits and non-finite states fail closed. The packed Triton implementation may hand a small active batch to the matched single-problem solver when the width falls below `min_packed_width`; the MetroPT-3 and Packer19 configurations set this threshold to one.

When one problem fits on an accelerator, windows are assigned to independent workers, one per GPU. Each device then has an independent update path and no per-update Sinkhorn collective. The scale-out experiments use this configuration; row sharding is reserved for an oversized problem.

4 Hardware-Aware Performance Model

The model combines per-problem completion depths with measured update time at each active batch width. Completion depths determine how many updates can be removed; width-dependent kernel time determines the runtime reduction. This follows the standard use of accelerator performance models to relate workload shape and hardware execution cost [9, 19].

4.1 Counting removed updates

For problem t , let n_t be the batched-iteration number of the first configured verification round at which the problem passes the two-marginal verifier, and let

$$A_k = |\{t : n_t \geq k\}| \quad (6)$$

be the active batch width before packed update k . Verification supplies both quantities.

Proposition 1 (Update-count identity). *For a finite window of W problems,*

$$S_{\text{active}} = \sum_{k=1}^{n_{\max}} A_k = \sum_{t=1}^W n_t, \quad S_{\text{static}} = W n_{\max}. \quad (7)$$

Hence dynamic compaction removes exactly

$$R = W n_{\max} - \sum_{t=1}^W n_t \quad (8)$$

problem updates relative to a fixed-width batch with the same final boundary.

Proof. Use $n_t = \sum_{k \geq 1} \mathbf{1}\{n_t \geq k\}$ and exchange the two finite sums. Fixed-width execution keeps all W lanes through $n_{\max}$. $\square$

Logical masking changes which states update but not the width seen by the kernel, so it retains $W n_{\max}$ updates. The area between the static rectangle and the active-width trajectory is therefore the exact work removed by dynamic compaction. The verification interval is already reflected in the observed n_t ; the first passing iteration on a denser verification schedule would define a different quantity.

Proposition 2 (Condition for removable work). *Let $(n_1, \dots, n_W)$ be any random vector of finite iteration numbers. Then $\mathbb{E}[R] > 0$ if and only if there exists a batched iteration k such that*

$$\Pr(0 < A_k < W) > 0. \quad (9)$$

Proof. R is nonnegative and is zero exactly when every lane has the same iteration number. Different finite iteration numbers create a value of k with both active and completed lanes, and the converse follows directly from the definition of A_k . $\square$

The condition depends on the joint distribution of iteration numbers inside a packed window. Perfectly synchronized lanes create no removable work even when the required iteration numbers vary across windows.

4.2 Exact wall-clock accounting for a measured backend

Let $c_k(a, \xi_k)$ be the cost of matched update k at width a , where ξ_k records support shape, data type, kernel

configuration, and other backend state held fixed by the comparison. Then

$$T_{\text{active}} = \sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{verification}} + H_{\text{compact}} + H_{\text{other}}, \quad (10)$$

H_{other} covers remaining timed control work. Let H_{fixed} be the corresponding fixed-width control cost and let ΔH denote active control cost minus H_{fixed} .

Proposition 3 (Runtime decomposition). *For a matched fixed-width/dynamic-compaction execution with the same final verifier boundary and matched update states ξ_k ,*

$$T_{\text{fixed}} - T_{\text{active}} = \sum_{k=1}^{n_{\max}} [c_k(W, \xi_k) - c_k(A_k, \xi_k)] - \Delta H. \quad (11)$$

Consequently, dynamic compaction is faster exactly when

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > \Delta H. \quad (12)$$

Proof. Write $T_{\text{fixed}} = \sum_k c_k(W, \xi_k) + H_{\text{fixed}}$, subtract Equation (10), and collect the control terms. $\square$

Variation among n_t creates removable updates. The measured function $c_k(a, \xi_k)$ determines whether the smaller active widths reduce kernel time. If $c_k(A_k, \xi_k) = c_k(W, \xi_k)$ for every k , compaction removes updates but saves no update time. It reduces total runtime when the summed update-time reduction exceeds ΔH .

4.3 Width-dependent kernel time

For the packed Triton update used in our implementation, the launch grid has

$$B(a, n) = a \left\lceil \frac{n}{b_m} \right\rceil \quad (13)$$

thread blocks, where b_m is the row tile size. GPU scheduling processes this grid in occupancy-dependent waves [9, 10]. The resulting model is

$$c(a; n, \xi) \approx g_{n, \xi} \left(\left\lceil \frac{B(a, n)}{Q_\xi} \right\rceil \right), \quad (14)$$

where Q_ξ is the measured resident block capacity of the kernel and $g_{n, \xi}$ maps wave count to time. Registers, shared memory, and kernel state enter through Q_ξ ; the value is calibrated for the selected device and kernel configuration.

On the measured A100/Triton configuration with $b_m = 64$, the first-wave estimate places the saturation width near 6.75 for $n = 1024$, 1.69 for $n = 4096$, and below one for $n = 16384$. Direct primitive measurements locate the first large increase at widths $6 \rightarrow 7$ and $1 \rightarrow 2$ for the first two sizes; the $n = 16384$ curve is responsive from width one. These measurements instantiate $c(a)$ for this backend and explain why equal update-count reductions can have different wall-clock value across problem sizes.

5 Experiments

5.1 Experimental setup

We evaluate four named tasks: (1) **MetroPT-3** industrial sensor streams, (2) **ImageNet-32** flow-matching training, (3) **Packer19** single-cell trajectory inference, and (4) **A2D2** LiDAR point-cloud matching.

Timing levels. We report three nested measurement scopes. E_1 (**Sinkhorn time**) ends at the verified coupling or dual potentials. E_2 (**pipeline time**) adds the fixed downstream consumer. E_3 (**total time**) covers the complete model-training, generation, or task-evaluation interval. Speedup is always baseline time divided by DrainSinkhorn time. If E_1 is reconstructed from an E_2 record, the paired consumer time is subtracted within each timing unit before forming ratios or uncertainty summaries.

Comparisons and controls. Fixed-width/dynamic-compaction compares the conventional batch with the full DrainSinkhorn execution path, including release and physical compaction. Logical-mask/dynamic-compaction (LM/DC) holds the observed release decisions fixed and isolates the effect of narrowing the batch. Matched component tests hold the input, proposal, stopping tolerance, verification schedule, two-marginal verifier, backend, and downstream consumer fixed. When stopping controllers reach different residuals, we report achieved residuals and consumer-output deviations with the timing result.

Workloads and numerical settings. The main packed-Triton studies use NVIDIA A100-SXM4-80GB GPUs and cold initialization. MetroPT-3 uses $(n, W, \varepsilon) = (16384, 16, 0.1)$, verification interval 4, and $\tau = 10^{-3}$. Packer19 uses $(16384, 8, 0.1)$ with the same interval; its component tests use $\tau = 10^{-3}$ and its stopping study sweeps 10^{-4} – 10^{-2} . ImageNet-32 uses PCA500 couplings with $(1024, 8, 0.03)$, interval 4, and $\tau = 10^{-3}$ for three paired 50,000-update training runs evaluated at NFE 4, 8, and 16. The A2D2 transfer test uses PyKeOps at $(n, W) = (8192, 8)$ over three real LiDAR clips. Appendix Table 8 gives the endpoint and statistical unit for each main workload, and Appendix Table 10 records the timing scope and implementation of every attribution contrast.

Section 5.2 measures application runtime across tasks. Section 5.3 measures packed-kernel time as the active batch width changes. Section 5.4 isolates verification, screening, and physical compaction on Packer19. Section 5.5 changes MetroPT-3 grouping, and Section 5.6 separates numerical safety from scaling across independent GPU workers.

5.2 Application-level results and backend transfer

Table 2 summarizes the matched comparisons by initial width.

The matched comparisons begin from conventional fixed-width batched execution. The primary MetroPT-3 study compares the two paths on one host with one worker per GPU. Dynamic compaction gives $1.746\times$ on the pipeline endpoint while the number of problem updates falls from 5696 to 3128. All three prespecified execution orders favor dynamic compaction, with a descriptive range of $1.7453\text{--}1.7460\times$. The corresponding four-GPU energy ratio is $1.779\times$.

A separate two-host MetroPT-3 study gives a fixed-width/dynamic-compaction ratio of $1.744723\times$ on the E_2 endpoint and $1.754\times$ at E_1 after subtracting the paired consumer within each host-plan unit. Its six host-plan units all favor dynamic-compaction execution.

The MetroPT-3 scale-out study uses eight GPU workers on two hosts. Its two fixed-work-per-worker configurations give $3.110\times$ and $2.159\times$ matched fixed-width/dynamic-compaction speedups while reducing updates as shown in table 3. Separate single-worker support-size configurations that meet the prespecified criteria on both hosts give $2.316\times$ at $n = 8192$ and $1.616\times$ at $n = 32768$.

In the ImageNet-32 training study, each step uses a real PCA500 feature coupling. Table 9 reports the four solver-phase times. The paired Sinkhorn-time ratio is $1.054\times$ (observed seed range $1.033\text{--}1.072\times$; table 11). Over the full fixed-update training interval, the ratio is $1.036\times$ (range $1.030\text{--}1.043\times$). A separate ImageNet-32 diagnostic measures coupling preparation over three problem-plan seeds, with fixed-width/dynamic-compaction $1.105\times$. The study that uses one initialization for multiple problems supplies a non-cold setting: across 40 controlled overlapping-support windows, compacted execution is $1.421\times$ faster, reduces updates by 30.62%, and wins every paired window.

The quality results use the same three paired training seeds at NFE 4, 8, and 16. The full per-seed measurements and analysis artifacts are available in the public GitHub repository at <https://github.com/cis-aceticacid/DrainSinkhorn>. The evaluation measures the ImageNet-32 PCA500 feature-space OT-FM endpoint.

5.3 Kernel time by active batch width

To explain when removed lanes become wall-clock savings, we measure the width-cost term in equation (11). The diagnostic calls the same packed Triton update at fixed problem size and varies only the width.

As shown in figure 2, the measured medians define $g_W(a) = c(W, \xi)/c(a, \xi)$ for static widths $W = 8$ and 16, using the common active-width coordinates $a \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. For a realized active width A_k , the corresponding summand in equation (11) is $c(W, \xi_k)[1 - g_W(A_k)^{-1}]$. The panel value $\Delta = b_m N_{SM}/n$ predicts the plateau scale: wave quantization is coarse

at $n = 1024$ ($\Delta = 6.75$), becomes finer at $n = 4096$ ($\Delta = 1.69$), and is near continuous at $n = 16384$ ($\Delta = 0.42$).

The diagnostic estimates the packed-update component of $c_k(a, \xi_k)$. Its $n = 1024, W = 8$ shape is consistent with the ImageNet-32 Sinkhorn-time ratio: after the active width enters the $A \leq 6$ region, further lane removal changes the update count more than packed-update time. The $n = 16384$ response provides the corresponding kernel-time behavior associated with the larger MetroPT-3 gains; the matched endpoint measures the net effect, including control and consumer work.

Transfer across backend families. The OTT-JAX and PyKeOps rows test the execution principle with independent implementations. OTT-JAX execution that processes only the remaining problems in each fixed phase is $2.600\times$ faster than native OTT-JAX on real ImageNet-32 at $W = 16$; its matched fixed-width/dynamic-compaction control assigns $1.545\times$ to removing completed problems only at phase boundaries and then rebucketing the remaining problems. On real A2D2 LiDAR, the PyKeOps compacted path is $3.174\times$ faster than sequential carry at $W = 8$. The ImageNet-32 PyKeOps test gives $1.415\times$. The tiled-log implementation reaches $n = 65536$ without materializing per-problem cost tensors and supplies the measured capacity boundary.

Independent OTT-JAX and PyKeOps realizations are summarized in table 7.

5.4 Packer19 single-variable component tests

The component study uses one Packer19 transition workload on two hosts, each with four A100-SXM4-80GB GPUs. Three prespecified method orders produce 24 paired observations indexed by host, order, and GPU. Every observation uses the same input, packed-kernel source tree, 10^{-3} stopping tolerance, two-marginal verifier, and transition consumer; timings are warmup-excluded repeat medians. Each row of Table 5 changes exactly one execution component under this shared measurement boundary.

Table 2: Matched results grouped by initial width. Speedup is baseline time divided by dynamic-compaction time. Brackets give registered 95% intervals and parentheses give observed seed or plan ranges. Rows retain their study-specific timing scopes and are not pooled. Row-level provenance is recorded in `support/figure_materials/MAIN_RESULTS_BY_WIDTH.csv`.

W	Campaign	Configuration	Contrast	Speedup	Timing scope and statistical unit
8	ImageNet-32	$n = 1024$	fixed/DC	$1.054 \times (1.033\text{--}1.072)$	Sinkhorn time; 3 training seeds
8	ImageNet-32	$n = 1024$	fixed/DC	$1.105 \times (1.079\text{--}1.156)$	coupling preparation; 3 problem-plan seeds
8	Packer19	$n = 16384$	LM/DC	$1.5397 \times [1.5389, 1.5404]$	consumer-subtracted OT stage; 24 paired units
8	Packer19	$n = 16384$	LM/DC	$1.5713 \times [1.5701, 1.5726]$	direct OT stage at 10^{-3} ; 24 paired units
8	MetroPT-3	$n = 16384$, lower-workload input	fixed/DC	$2.159 \times$	eight-worker pipeline endpoint; fixed work per worker
8	MetroPT-3	$n = 16384$, higher-workload input	fixed/DC	$3.110 \times$	eight-worker pipeline endpoint; fixed work per worker
16	ImageNet-32	$n = 4096$, shared anchor	fixed/DC	$1.421 \times$	steady-state critical path; 40 windows
16	ImageNet-32	$n = 4096$, OTT-JAX	fixed/DC	$1.581 \times$	matched 20-update phases; 10 repeats
16	MetroPT-3	$n = 32768$, lower-workload input	fixed/DC	$1.616 \times$	one-worker endpoint; equal-host geometric mean
16	MetroPT-3	$n = 16384$, two hosts	fixed/DC	$1.7447 \times [1.7434, 1.7467]$	pipeline endpoint; 6 host-plan observations
16	MetroPT-3	$n = 16384$, four workers	fixed/DC	$1.7455 \times (1.7453\text{--}1.7460)$	four-worker makespan; 3 registered plans
16	MetroPT-3	$n = 8192$, lower-workload input	fixed/DC	$2.316 \times$	one-worker endpoint; equal-host geometric mean

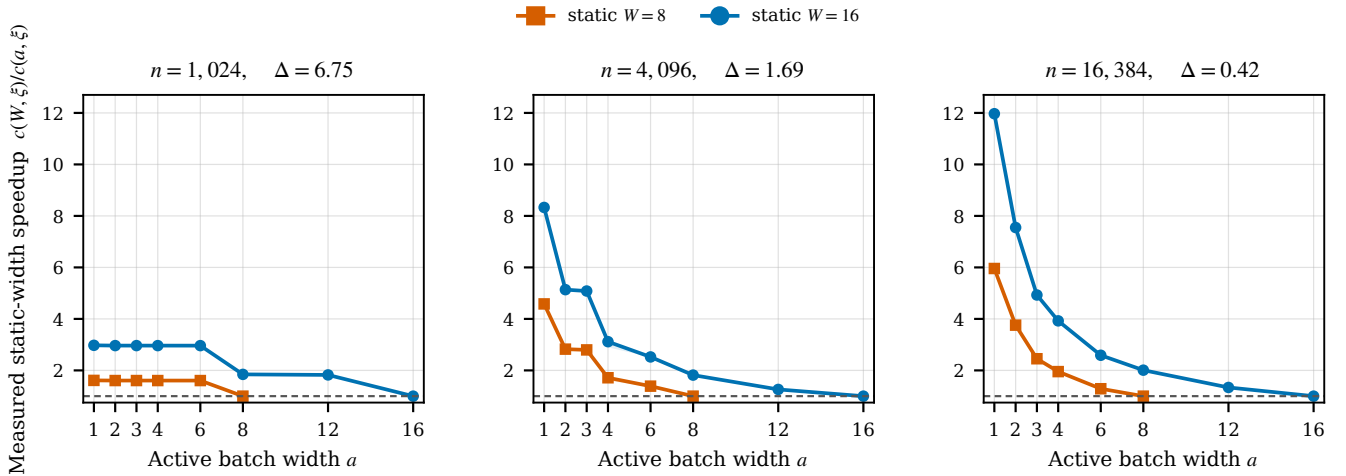


Figure 2: Packed-update speedup by active batch width. Each curve reports $g_W(a) = c(W, \xi)/c(a, \xi)$ relative to static width $W \in \{8, 16\}$; all panels use the same axes, and shading propagates the per-width interquartile endpoints from 30 repeats. Panel titles report $\Delta = b_m N_{SM}/n$, which determines scheduling-wave quantization. The $(n, W, \varepsilon) = (1024, 8, 0.03)$ setting matches the ImageNet-32 training solver configuration, which verifies every four updates at tolerance 10^{-3} ; the plotted metric is packed-update time.

Table 3: MetroPT-3 fixed-work execution on eight GPU workers across two hosts. Times cover the pipeline endpoint. Counts report problem updates across workers; energy uses the same endpoint.

Input	Fixed (s)	Dynamic (s)	Updates fixed/dynamic	Time	Energy
Higher workload	12.523	4.026	4736 / 1276	$3.110 \times$	$3.437 \times$
Lower workload	7.999	3.705	3008 / 1244	$2.159 \times$	$2.292 \times$

Table 4: ImageNet-32 PCA500 feature-space quality after 50k training updates. Entries are three-seed means for matched fixed-width (Fixed) and dynamically compacted (Dynamic) execution. Lower is better for all three metrics. The same three paired training seeds are used throughout the quality evaluation; the full per-seed measurements and analysis artifacts are available in the public GitHub repository. These are feature-space metrics, not pixel-space FID.

NFE	Curvature		Held-out FM MSE		Projected Fréchet	
	Fixed	Dynamic	Fixed	Dynamic	Fixed	Dynamic
4	5.4524	5.4448	1.07854	1.07859	1.2967	1.2886
8	2.8388	2.8356	1.07854	1.07859	1.5286	1.5224
16	1.4546	1.4528	1.07854	1.07859	1.8182	1.8136

Table 5: Packer19 single-variable attribution under one frozen input and stopping rule. Every row reports OT-stage time from a paired comparison after subtracting consumer time, so the component effects share one measurement boundary.

Change	Matched contrast	Ratio and evidence
Batched verification	serial / batched verifier	1.133×; 24/24 wins
Current-marginal screen	verifier only / preliminary screen	1.105×; 24/24 wins
Logical masking	fixed / logical mask	1.000×; 95% CI [0.9998, 1.0003]
Dynamic compaction	logical mask / dynamic compaction	1.540×; 95% CI 1.5389–1.5404×

The batched-verification and current-marginal-screen rows have 24/24 paired wins; no separate E_1 bootstrap interval is registered for those two contrasts.

Verification and screening. As shown in Table 5, batched evaluation of the two-marginal verifier and the preliminary one-marginal screen reduce OT-stage time under otherwise matched paths. Each ratio subtracts its paired consumer time before forming the contrast. To test whether screening delays a release, we replay all 1488 screen-negative events with the full two-marginal verifier. The replay finds 0 already-releasable lanes; the maximum screen-verifier row-residual difference is 1.86×10^{-7} .

Physical compaction. Logical masking leaves the packed kernel at width eight and has no measurable effect in the matched control. Dynamic compaction removes 100 of 256 problem updates and yields $1.540\times$ OT-stage acceleration with interval $1.5389\text{--}1.5404\times$; all 24 paired units favor compaction. Across the paired-observation medians, endpoint energy falls from 879.9 to 688.1 J, while peak allocated memory rises from 80.8 to 106.9 MiB because of the compacted buffers. Phase costs are reported separately in table 12.

Stopping-tolerance sweep. The two-host stopping study repeats the LM/DC comparison at 10^{-4} , 3×10^{-4} , 10^{-3} , 3×10^{-3} , and 10^{-2} without pooling across tolerances. All arms use the same two-marginal verifier, residual tolerance, and tight-reference consumer output.

Dynamic compaction improves over logical masking by $1.480\text{--}1.638\times$ from 10^{-4} through 3×10^{-3} . At 10^{-2} , all eight problems first pass at update 8, leaving no removable padding; LM/DC is therefore neutral at $0.9995\times$ (95% CI $[0.9987, 1.0003]$).

5.5 MetroPT-3 grouping test

The grouping test holds a set of 32 real MetroPT-3 problems fixed and changes only their assignment to windows. Offline every-cycle iteration numbers q_t define the grouping, while verifier-derived iteration numbers n_t measure executed work. Oracle homogeneous grouping minimizes variation in the required iteration numbers within each window. Heterogeneous and random groupings increase the number of updates executed after some problems have completed. As shown in Table 6, the matched fixed-width/dynamic-compaction ratio grows from $1.105\times$ to $1.217\times$ and $1.235\text{--}1.271\times$ on random layouts.

Table 6: MetroPT-3 grouping intervention on one fixed real task multiset. For offline every-cycle iteration numbers q_s within a window, $H_q = \text{sd}(q_s)/\text{mean}(q_s)$ and padding is $W \max_s q_s / \sum_s q_s - 1$. The dynamic-update ratio divides the dynamic update count by the fixed-width count. Values aggregate over windows; timing repeats stabilize each condition and are not independent datasets.

Grouping	H_q	Padding	Dynamic updates	Fixed/dynamic
heterogeneous	0.236	0.236	0.802	$1.217\times$
homogeneous	0.127	0.110	0.892	$1.105\times$
random 1	0.254	0.293	0.776	$1.258\times$
random 2	0.264	0.270	0.789	$1.235\times$
random 3	0.260	0.279	0.764	$1.271\times$

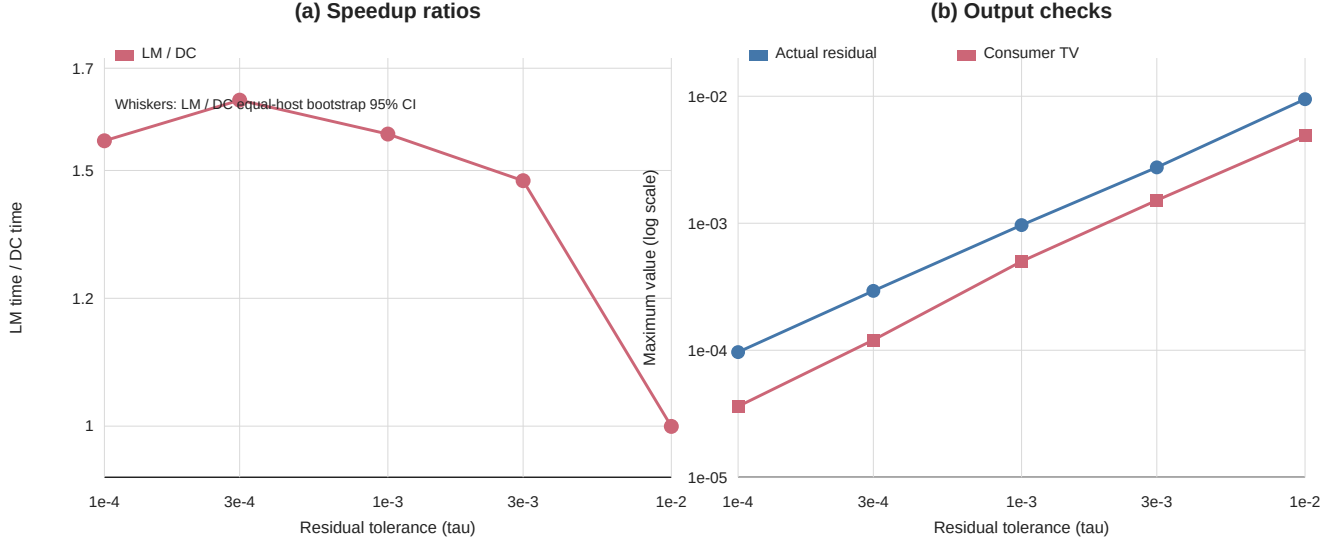


Figure 3: Two-host Packer19 stopping-execution results under a common two-marginal verifier and residual tolerance. Panel (a) shows the LM/DC speedup ratio across the five tolerances; whiskers are equal-host stratified bootstrap 95% intervals. Panel (b) shows maximum actual residual and maximum consumer TV on a logarithmic scale. Each point summarizes 24 paired units; consumer TV measures output differences against the tight reference.

5.6 Numerical safety and scale-out

Independent finite-precision stress test. An independent stress test measures decision behavior on 84 near-threshold problems and 16 runtime attacks over 1/2/4/8 GPUs. Relative to independent FP64 replay, raw FP32 and TF32 each disagree on three accept decisions; an outward-bound guard followed by FP64 escalation has zero disagreements in these 100 cases. This guarded configuration is evaluated separately from the timed applications.

Timed applications and scaling across GPU workers. The timed packed Triton path uses the precision supported by the backend and the two-marginal verifier. All emitted plans meet both recorded marginal-residual bounds, and every endpoint passes its output or quality check. The MetroPT-3 scale-out study assigns complete windows to independent GPU workers. Four workers achieve $3.965\text{--}3.979\times$ fixed-per-worker throughput scaling on held-out MetroPT routes, with no Sinkhorn collective between workers; the eight-worker matched endpoint decomposition appears in table 3.

6 Operating Regime and Composition

DRAINSINKHORN applies to batched positive EOT problems with compatible batched representations and a two-marginal verifier. Different numbers of iterations within a batch create removable work; net acceleration depends on

runtime as a function of batch width and on incremental control and state-movement costs (equation (12)).

Initialization shapes both the total iteration count and its variation within a window. The study that uses one initialization for multiple problems uses controlled overlapping ImageNet-32 supports: dynamic-compaction execution reduces updates by 30.62% and accelerates the steady-state critical path by $1.421\times$, including proposal and transfer costs. The feature-training study uses cold proposals for independently resampled OT-FM couplings.

The executor removes problems under the configured stopping rule. The grouping test changes how a fixed problem set is assigned to batches using offline iteration counts. The width sweep measures update time at each active batch width. These measurements supply the terms in equation (11) and can also guide grouping when completion depths can be estimated.

Dynamic compaction trades computation for state movement and buffers. The matched logical-mask/dynamic-compaction study reports 26.1 MiB of additional peak allocation alongside lower time and endpoint energy; table 12 reports its phase costs. OTT-JAX and PyKeOps realize release through backend-specific rebucketing and batched independent-problem execution.

The measured width-runtime relation belongs to a specified device, kernel path, problem shape, and numerical configuration. The packed Triton path also switches narrow tails to its matched single-problem solver, so its complete runtime includes that routing boundary. The update-count identity and equation (11) apply with any measured width-runtime relation; the reported wave boundaries are specific to the A100 Triton configuration.

7 Conclusion

DRAIN SINKHORN removes padded OT work from batches whose problems require different numbers of iterations. It starts from fixed-width execution of independent Sinkhorn problems. The solver’s stopping rule determines completion, and dynamic compaction narrows each subsequent update. The hardware-aware performance model combines an update-count identity with measured kernel time at each active width. Its runtime decomposition includes control overhead and gives the matched-path break-even condition.

Matched fixed-width/dynamic-compaction comparisons deliver $1.746\times$ on the four-worker MetroPT-3 endpoint, $1.054\times$ for ImageNet-32 Sinkhorn time, and $1.616\text{--}3.110\times$ in the MetroPT-3 scale-out study. A setting that uses one initialization for multiple problems gives $1.421\times$ with 30.62% fewer problem updates. OTT-JAX and PyKeOps implementations remove completed problems through independent backends.

Matched controls separate verification, screening, logical masking, and dynamic compaction. In MetroPT-3 and Packer19, compaction also reduces measured GPU energy. The grouping test shows that variation in completion depth creates more removable updates. A100 width sweeps show stepwise kernel times from wave quantization. The application results report runtime together with residuals and consumer or training outputs.

Reproducibility. The release artifact records source and input identities, commands, software and GPU environments, raw outputs, numerical checks, and paired analyses.

References

- [1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html.
- [2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL <https://proceedings.mlr.press/v202/amos23a.html>.
- [3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL <https://arxiv.org/abs/1306.0895>.
- [4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL <https://arxiv.org/abs/2201.12324>.
- [5] Jean Feydy, Thibault Sjourner, Franois-Xavier Vialard, Shun ichi Amari, Alain Trouv, and Gabriel Peyr. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL <https://proceedings.mlr.press/v89/feydy19a.html>.
- [6] Rmi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurlie Boissunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Lo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL <https://www.jmlr.org/papers/v22/20-451.html>.
- [7] Ginkgo Project. Batched CG. Ginkgo user guide, 2026. URL <https://gko-project.org/user-guide/concepts/batched/cg.html>. Accessed 2026-09-08.
- [8] Mete Kemerts, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL <https://arxiv.org/abs/2504.02067>.
- [9] *GPU Performance Background User’s Guide*. NVIDIA, 2022. URL <https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html>.
- [10] *CUDA C++ Programming Guide*. NVIDIA, 2024. URL <https://docs.nvidia.com/cuda/archive/12.5.0/cuda-c-programming-guide/index.html>.
- [11] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL <https://proceedings.mlr.press/v202/pooladian23a.html>.
- [12] POT developers. POT 0.9.6 release notes. Python Optimal Transport documentation, 2025. URL <https://pythonot.github.io/releases.html>.
- [13] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.
- [14] Meyer Scetbon, Marco Cuturi, and Gabriel Peyr. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL <https://proceedings.mlr.press/v139/scetbon21a.html>.

- [15] Alexis Thibault, Lénaïc Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL <https://arxiv.org/abs/1711.01851>.
- [16] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL <https://proceedings.mlr.press/v206/thornton23a.html>.
- [17] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL <https://arxiv.org/abs/2606.26341>.
- [18] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL <https://arxiv.org/abs/2602.03067>.
- [19] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 503–521, 2021. URL <https://www.usenix.org/conference/atc21/presentation/yu>.
- [20] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [21] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL <https://arxiv.org/abs/2506.05526>.

A Measurement Contracts and Additional Detail

A.1 ImageNet-32 seed provenance

The quality evaluation uses the same three paired training seeds, 20260891, 20260892, and 20260893, throughout the reported NFE values. The full per-seed measurements and analysis artifacts are available in the public GitHub repository at <https://github.com/cis-aconiticacid/DrainSinkhorn>.

Measurement levels. E_1 terminates at the measured coupling output or potentials after the two-marginal verifier passes. E_2 includes the fixed downstream consumer. E_3 includes training, generation, or task evaluation. The MetroPT-3 and Packer19 pipeline studies record E_2 endpoints, whereas ImageNet-32 records an E_3 training endpoint. The primary MetroPT-3 ratio

uses its registered E_2 endpoint directly. The two-host MetroPT-3 and Packer19 component studies subtract each unit’s paired consumer time before ratio formation and bootstrap, while ImageNet-32 uses the recorded Sinkhorn time. The Packer19 stopping study records E_1 directly for every stopping-controller and execution-mode cell. Unchanged consumer or model time in E_2 or E_3 dilutes the corresponding OT-stage gain. The ImageNet-32 fixed-width/dynamic-compaction ratio reported as a training ablation is computed from the full fixed-update E_3 interval.

GPU-energy interval. GPU energy is the change in the A100 NVML cumulative total-energy counter from immediately before the registered method call to after the final CUDA synchronization. The counter reports millijoules and the analysis converts the difference to joules. This pipeline interval starts with prepared input arrays and ends after the downstream consumer. It includes solver/proposal and consumer work and excludes compilation, warm-up, and the prepared-input transfer component. Unsupported counters are recorded as unavailable rather than replaced by a power-limit estimate.

Primary MetroPT-3 matched fixed/dynamic control. This study runs fixed-width and dynamic-compaction paths on the same MetroPT-3 input, cold proposal, regularization, tolerance, verification schedule, two-marginal verifier, packed backend, and fixed consumer. The experimental variable is whether verifier-approved lanes are dynamically removed from later kernels. One host supplies four independent A100 workers, one per GPU, with no inter-worker collective or problem sharding. For each plan and timing repeat, the endpoint is the maximum pipeline time across the four workers. Each plan is summarized by the median of two timing repeats, and the reported $1.745535\times$ is the descriptive geometric mean of the three plan ratios. The plan identifiers freeze execution order and are not independent MetroPT samples. All 48 worker-level repeats converge; the maximum row and column L_1 residuals are 9.476650×10^{-4} and 1.283915×10^{-6} . Both arms attain failure-window ROC AUC 1.0 under the frozen consumer. The solver receives sensor inputs without failure labels. The largest paired transport-cost and barycentric-shift differences are 0.02792263 and 0.00346756.

Two-host MetroPT-3 matched control. Within this matched contrast, fixed-width and dynamic-compaction execution use the same inputs, cold proposal, regularization, tolerance, verification schedule, two-marginal verifier, packed backend, and fixed consumer; stopping-rule-triggered release and dynamic compaction are the controlled difference. Three order-balanced plan seeds are reused across four GPU placements on each of two hosts. GPU placements and timing repeats stabilize end-to-end measurement; the registered endpoint ratio in-

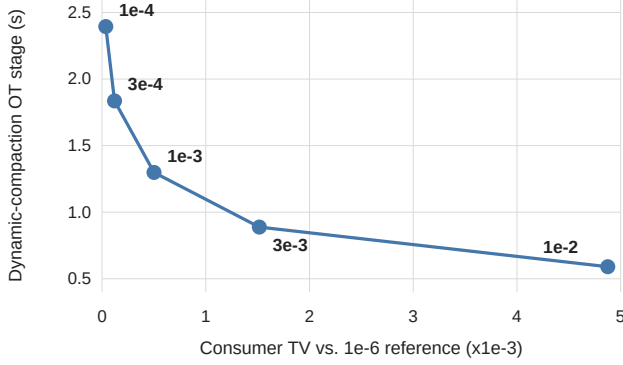


Figure 4: Tolerance-dependent Packer19 OT speed-output frontier for the dynamic-compaction path. Each point is the median E1 OT-stage time over 24 paired units at that residual tolerance; consumer TV is measured against the campaign’s 10^{-6} tight reference.

cludes unchanged non-OT consumer time. The main-text MetroPT-3 OT-stage ratio subtracts the paired consumer time within each host-plan unit. The host-plan combinations are paired timing units for this fixed workload; placements and repeats are timing replicas. A reanalysis of the six valid host-plan units reports a stratified bootstrap interval.

Packer19 stopping-study provenance. The stopping-execution protocol has SHA-256 `c010bcf9b3e276c8269514324cc8ed4bc2fc5958c362c7b880e3946dee75075b` and is bound to source commit `96de52b8ceff8fa3c0468bf45567e6b017483e73`. Two hosts, four A100-SXM4-80GB GPUs per host, three prespecified method orders, and three timing repeats produce 768 formal cells. Each tolerance has 24 paired paired observations indexed by host, order, and GPU and an equal-host bootstrap interval. Consumer TV numerically checks differences in outputs.

Execution-mode protocol. The experiments fix representation, width, execution mode, verification interval, stopping tolerance, verification path, tail-handoff threshold, and fail-closed policy before timing. The width and grouping interventions map conditions for selecting fixed-width execution or dynamic compaction.

Width-cost diagnostic provenance. The primitive sweep uses one NVIDIA A100-SXM4-80GB with 108 SMs, driver 535.129.03, PyTorch 2.5.1 with CUDA 12.4, Triton 3.1.0, and source commit `73aec2941bfa1a5caa6a6bddf7aa83b734f9bb8b`. Each measured call to `shifted_step` contains two packed Triton log-sum-exp kernels and excludes the marginal screen, two-marginal verifier, dynamic compaction, and outer `solve_batch` loop. Widths are randomized within 30-repeat blocks after warm-up. The $n = 1024$ sweep covers widths 1–8, 10, 12, and 16; the

$n = 4096$ and $n = 16384$ sweeps cover 1, 2, 3, 4, 6, 8, 12, and 16. This diagnostic supplies the packed-update component of the measured width-cost curve in equation (11).

Independent backend realizations. The OTT-JAX and PyKeOps paths remove completed problems through their native backend mechanisms. OTT-JAX executes fixed 20-update phases, verifies at phase boundaries, removes passing lanes, and rebuckets the remaining problems; native `vmap` and a matched fixed-phase static path provide its controls. The PyKeOps paths use one batched independent-problem matrix-free operator, a fixed geometric verification schedule, the two-marginal verifier, and removal of passing problem state. Sequential and static-padded PyKeOps paths use the same symbolic backend. The shared principle is per-problem release followed by dynamic removal; verifying and packing mechanisms remain backend-specific. All cross-backend rows in table 7 report their achieved two-marginal residual.

A.2 Independent backend implementations

B Deployment and Operating Conditions

One worker per GPU and row partitioning. When one problem fits on one accelerator, GPU workers process different windows and require no Sinkhorn collective. Partitioning the matrix by rows supplies a capacity path for an oversized single problem. One-worker-per-GPU execution is the throughput path used by the MetroPT-3 scale-out study. On one host, increasing from one to four workers yields $3.965\text{--}3.979\times$ fixed-per-worker throughput scaling on two held-out MetroPT routes. The two-host, eight-worker fixed-per-worker run reports endpoint comparisons at that worker count. We report the one-to-four and two-host eight-worker results as distinct scaling studies. Peak memory remains local to each worker.

B.1 Supplementary attribution and evidence boundaries

The ImageNet-32 coupling-preparation diagnostic holds the packed Triton adapter, input, and numerical contract fixed and changes dynamic compaction; its three fixed-width/dynamic-compaction ratios are 1.156, 1.082, and 1.079. Fixed-width/dynamic-compaction update counts are 64/52, 64/56, and 64/56. The measurements cover coupling preparation. The OTT-JAX study uses one frozen ImageNet-32 input and ten timing repeats per tolerance; the phase size is chosen on a disjoint warmup bank. Its ratio at 10^{-2} is $0.995\times$.

The MetroPT-3 eight-worker results aggregate the downstream-consumer endpoint at fixed work per worker.

Table 7: Independent OTT-JAX and PyKeOps realizations on real data. Every ratio is baseline time divided by the compacted path, and every uncertainty label states whether it is a repeat or observed-input range.

Backend	Workload	OT-stage contrast	Speedup (uncertainty)	Timing; unit	Accuracy contract
OTT-JAX	ImageNet-32, $n = 4096, W = 16$	native vmap / remaining problems processed in each fixed phase	$2.600\times$ (repeat range $2.338\text{--}2.664\times$)	direct E1; 6 timing repeats	max L1 9.97×10^{-4}
PyKeOps	A2D2 LiDAR, $n = 8192, W = 8$	sequential carry / active	$3.174\times$ (clip range $2.180\text{--}3.944\times$)	direct E1; 3 real clips	max L1 9.97×10^{-4}
PyKeOps	ImageNet-32, $n = 16384, W = 4$	sequential / active	$1.415\times$ (seed range $1.360\text{--}1.448\times$)	direct E1; 3 seeds	max L1 8.93×10^{-4}

Table 8: Configuration of the main workloads. All rows use cold initializations and a verifier that checks both current marginal residuals.

Benchmark	Endpoint, numerical configuration, and statistical unit
MetroPT-3, four-worker	Predictive maintenance (E_2); $(n, W, \varepsilon) = (16,384, 16, .1)$; verification interval 4; $\tau = 10^{-3}$; one host, one worker per GPU, three execution orders, two timing repeats.
MetroPT-3, two-host	Predictive maintenance (E_2); $(n, W, \varepsilon) = (16,384, 16, .1)$; verification interval 4; $\tau = 10^{-3}$; two hosts and three plans.
Packer19 components	Single-cell transition (E_2); $(16,384, 8, .1)$; verification interval 4; $\tau = 10^{-3}$; 24 paired observations.
Packer19 stop-ping	Fixed-controller tolerance sweep (E_1); $(16,384, 8, .1)$; verification interval 4; $\tau = 10^{-4}\text{--}10^{-2}$; 24 paired observations per tolerance.
ImageNet-32	PCA500 OT-FM training (E_3); $(1,024, 8, .03)$; verification interval 4; $\tau = 10^{-3}$; three paired seeds and 50,000 fixed updates.

Fixed/dynamic seconds are 12.523/4.026 for the higher-workload input and 7.999/3.705 for the lower-workload input, with update counts 4736/1276 and 3008/1244. The support-size comparison uses the lower-workload input at $n = 8192$ and 32768 and applies the same prespecified criteria on both hosts. Transport-cost and barycentric-shift differences are diagnostics under the registered binary consumer endpoint.

The ImageNet-32 shared-anchor result uses a constructed 90%-overlap target stream over real ImageNet features. Its canonical reverse-order warm replay excludes compile, cache load and process startup, while retaining descriptor, transfer, anchor, proposal, correction and verification costs. The MetroPT-3 grouping ratios describe one fixed 32-problem multiset; oracle iteration numbers q_t define the grouping intervention. The fixed multiset fixes $\sum_t q_t$, while grouping changes $\sum_w W q_{\max,w}$ across windows. The ratios report the resulting relative-runtime effect.

Table 9: Absolute OT-stage time (seconds) for the three packed-Triton workloads at the 10^{-3} operating point. Packer19 reports the median over 24 paired observations indexed by host, order, and GPU. ImageNet-32 reports solver-phase mean $\pm$ sample standard deviation over three paired training seeds. MetroPT-3 reports the median of the three worker-1 execution-order observations. The Packer19 fixed-width arm executed the compaction branch and is excluded, so its value is unavailable. A dash marks an unavailable matched arm. These absolute summaries use the listed units; paired effects are reported separately.

Workload	Fixed-width batch	Logical mask	Dynamic compaction
Packer19 ($n = 16384, W = 8$)	—	2.038	1.299
MetroPT-3 ($n = 16384, W = 16$)	14.839	—	8.462
ImageNet-32 PCA500	122.17 ± 2.91	—	115.89 ± 0.83

Table 10: Evidence map for the reported attribution contrasts. Ratios are numerator-path time/denominator-path time; values above one favor the denominator. GM denotes geometric mean. Timing scope and sample/replication units are listed by study. The shared-anchor ImageNet-32 row uses controlled overlapping supports. Regularization is 0.03 for the PCA500 ImageNet-32 rows and 0.1 for the other rows.

Study	Backend / workload	$n, W; \tau$	Proposal	Contrast	Ratio	Timing scope	Sample / replication
MetroPT-3 four-worker	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	cold	fixed/dynamic	1.745535	E_2 four-worker makespan; updates $5696/3128 = 1.820972\times$	orders 1.745273 / 1.745356 / 1.745975; 2 repeats
MetroPT-3 two-host	packed Triton / MetroPT-3	$16384, 16; 10^{-3}$	cold	fixed/dynamic	1.754	E_1 ; paired consumer subtracted	host-plan units
ImageNet-32 solver	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/dynamic	1.054	Sinkhorn time; supplementary GM	3 training seeds
ImageNet-32 training	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/dynamic	1.036	full fixed-update training	3 training seeds
ImageNet-32 preparation	packed Triton / ImageNet-32	$1024, 8; 10^{-3}$	cold	fixed/dynamic	1.105	coupling preparation	3 problem-plan seeds
MetroPT-3 eight-worker	packed Triton / MetroPT-3	$16384, 8; 10^{-3}$	cold	fixed/dynamic	3.110 / 2.159	8-worker endpoint; fixed per worker	2 inputs; host/order repeats
MetroPT-3 support size	packed Triton / lower-workload input	$8192, 16; 10^{-3}$	cold	fixed/dynamic	2.316	one-worker endpoint; equal-host GM	2 hosts; 3 orders each
MetroPT-3 support size	packed Triton / lower-workload input	$32768, 16; 10^{-3}$	cold	fixed/dynamic	1.616	one-worker endpoint; equal-host GM	2 hosts; 3 orders each
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 10^{-3}$	fixed	fixed/dynamic	1.581	matched 20-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 3 \times 10^{-3}$	fixed	fixed/dynamic	1.556	matched 10-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 5 \times 10^{-3}$	fixed	fixed/dynamic	1.366	matched 10-update phases	one input; 10 repeats
ImageNet-32 OTT-JAX	OTT-JAX / ImageNet-32	$4096, 16; 10^{-2}$	fixed	fixed/dynamic	0.995	matched 10-update phases	one input; 10 repeats
ImageNet-32 shared anchor	packed Triton / ImageNet-32	$4096, 16; 10^{-3}$	shared anchor	fixed/dynamic	1.421	steady-state critical path	40 controlled windows
Packer19 fixed/mask	packed Triton / Packer19	$16384, 8; 10^{-3}$	cold	fixed/LM	1.000	E_1 ; paired consumer subtracted	24 paired observations
Packer19 mask/compact	packed Triton / Packer19	$16384, 8; 10^{-3}$	cold	LM/DC	1.540	E_1 ; paired consumer subtracted	24 paired observations
Packer19 tolerance sweep	packed Triton / Packer19	$16384, 8; 10^{-4} - 3 \times 10^{-3}$	cold	LM/DC	1.480–1.638	direct E_1 ; four tolerances	24 paired units/tolerance
Packer19 equal-depth control	packed Triton / Packer19	$16384, 8; 10^{-2}$	cold	LM/DC	0.9995	direct E_1 ; all problems finish together	24 paired observations

Table 11: ImageNet-32 Sinkhorn time and fixed/dynamic ratio by seed. The geometric mean of the three ratios is 1.054.

Seed	Fixed (s)	Dynamic (s)	Fixed/dynamic
20260891	123.429	116.699	1.058
20260892	124.233	115.934	1.072
20260893	118.840	115.036	1.033

Table 12: Packer19 phase times (seconds): marginal medians for each component of the matched logical-mask and dynamic-compaction implementations. Each phase is summarized separately; total-time ratios are computed from paired whole-path timings and reported in table 5.

Phase	LM (s)	PC (s)
Correction	1.7063	1.0708
Batched verification	0.6385	0.4006
Verifier	0.1045	0.1045
Mask / compaction	0.000306	0.003370